REAL-TIME DATA ANALYTICS • CAPSTONE PROJECT

Bank Loan Approval & Risk Analytics

A complete, runnable project: ingest applicant data, score credit risk with an explainable rule engine, train a machine learning approval model, and take decisions on a live stream of incoming loan applications.

DELIVERABLE	Student handbook + full source code + dataset
LANGUAGE / STACK	Python 3.9+ | pandas | scikit-learn | matplotlib | joblib
DATASET	20 historical applications + 8 live-stream applications (CSV)
RUN COMMAND	pip install -r requirements.txt → cd src → python main.py
LEVEL	Beginner to Intermediate | 8-12 hours

Everything needed to build and demonstrate this project is inside this zip file. No internet connection, database or API key required.

Table of Contents

1. Project Overview
   - Problem statement
   - Business context
   - What makes this project real-time
5. Objectives and Learning Outcomes
6. Project Scope
7. Requirement Specification
   - Functional requirements
   - Non-functional requirements
   - Software and hardware requirements
11. System Architecture
12. Dataset Description
   - Data dictionary
   - Sample records
   - Engineered features
16. Project Folder Structure
17. Methodology: The Five Stages
18. The Risk Scoring Rulebook
19. Code Walkthrough
   - config.py
   - data_loader.py
   - risk_engine.py
   - eda.py
   - model.py
   - stream_simulator.py
   - main.py
27. How to Run the Project
28. Expected Output
29. Results and Interpretation
30. Testing Checklist
31. Common Errors and Fixes
32. Viva / Interview Questions with Answers
33. Extension Ideas
34. Evaluation Rubric

35. Glossary

1 Project Overview

> Build a decision system that tells a bank, within milliseconds of an application arriving, whether to approve it, reject it or send it for manual review — and can always explain why it said so.

1.1 Problem statement

A retail bank receives loan applications continuously through its website, mobile app and branch counters. Each application must be assessed for **credit risk**: the probability that the borrower will fail to repay. Today that assessment is done manually by credit officers, which creates three problems:

- **Slow.** A manual review takes hours or days; customers abandon the application and go to a competitor.
- **Inconsistent.** Two officers looking at the same file can reach different conclusions.
- **Not measurable.** Nobody can see, in real time, how much money the bank has committed today or how risky today's portfolio is.

This project builds the software layer that solves all three: an automated, consistent, explainable and continuously monitored loan decision pipeline.

1.2 Business context

Two questions drive every lending business, and this project answers both:

Business question	How the project answers it
Should we lend to this person?	A rule engine produces a 0–100 risk score, and a Logistic Regression model produces a probability of approval. The two are combined into one final decision.
How exposed are we right now?	A rolling KPI tracker updates approval rate, average risk score, high-risk count and total sanctioned exposure after every single application.

1.3 What makes this project real-time

A batch analytics project loads a whole file, computes results once and stops. A real-time project is **event driven**: it waits for one record, reacts to it, updates its state, and waits again. This project reproduces that behaviour faithfully:

Real bank system	This project	Where in the code
Kafka / RabbitMQ / REST endpoint delivers one application	`incoming_stream.csv` is read row by row with a `time.sleep()` gap	`stream_simulator.run()`
Feature store computes ratios on arrival	`engineer_features()` runs on each record	`data_loader.py`
Policy engine applies regulator-approved rules	`score_application()` returns score, band and reasons	`risk_engine.py`
Deployed model returns a probability	The persisted pipeline is loaded once and called per record	`model.predict_one()`
Dashboard refreshes live metrics	`RollingKPI` recomputes and prints after every record	`stream_simulator.py`
Every decision is written to an audit log	Each decision is appended and saved to `realtime_decisions.csv`	`stream_simulator.run()`

Why simulate instead of using real Kafka

The event loop is deliberately isolated in one function. Replacing `for _, row in stream.iterrows()` with `for message in kafka_consumer` is the only change needed to make this production-grade — not one line of the scoring, model or KPI code would move. That separation is the real lesson.

2 Objectives and Learning Outcomes

2.1 Project objectives

#	Objective
O1	Ingest raw loan application data from CSV and validate its quality.
O2	Engineer the financial ratios lenders actually use (EMI, debt-to-income, loan-to-income).
O3	Implement a transparent, rule-based risk score from 0 to 100 with a written reason for every penalty applied.
O4	Explore the data statistically and visually to find what separates approved from rejected applications.
O5	Train, evaluate and persist a supervised classification model that predicts approval.
O6	Process a live stream of new applications, take a decision on each within the loop, and maintain rolling KPIs.
O7	Produce an auditable output set: charts, a saved model, a decision log and a summary report.

2.2 What you will learn

Area	Skills demonstrated
Python engineering	Modular packages, a single configuration module, docstrings, `if __name__ == "__main__"` test hooks, `pathlib`, `argparse` command-line flags
Data handling with pandas	Reading CSV, missing-value strategy, duplicate removal, `groupby` aggregation, `apply` with `result_type="expand"`, correlation analysis
Domain feature engineering	Reducing-balance EMI formula, debt-to-income ratio, loan-to-income multiple — the metrics real underwriters use
Machine learning	`ColumnTransformer`, `OneHotEncoder`, `StandardScaler`, `Pipeline`, stratified train/test split, accuracy, ROC-AUC, confusion matrix, coefficient interpretation, model persistence with joblib
Visualisation	Bar chart, box plot, bubble scatter plot, headless matplotlib rendering with the Agg backend
Real-time systems thinking	Event loops, per-record scoring, rolling aggregates, decision arbitration between a rule engine and a model, audit logging
Responsible AI	Why explainability matters in regulated lending, why a rule veto sits on top of the model, and how to spot a biased feature

3 Project Scope

In scope	Out of scope
CSV based ingestion of applicant data	Live connection to a core banking system or credit bureau API
Rule-based risk scoring with written justifications	Regulatory certification of the scorecard
Binary approval classification (approve / reject)	Loan pricing, interest rate optimisation or limit assignment
Simulated real-time stream driven by a CSV and a timer	A production message broker such as Kafka or RabbitMQ
Rolling KPIs printed to the console and written to CSV	A hosted, multi-user monitoring dashboard with alerting
A single Logistic Regression model	Deep learning, model ensembles or automated hyper-parameter search
Local file storage of all artefacts	Cloud storage, a data warehouse or a feature store
An optional Streamlit dashboard	Authentication, role-based access or deployment

Deliberately small dataset

Twenty training records is far below what a real scorecard needs (banks use hundreds of thousands). It is intentional: you can read every row, verify every calculation by hand, and still exercise the complete pipeline. Section 13 explains exactly which conclusions this sample size does and does not support, and Section 17 shows how to scale it up.

4 Requirement Specification

This is the contract the code must satisfy. Every requirement below is implemented, and the module that implements it is named so you can verify it yourself.

4.1 Functional requirements

ID	Requirement	Implemented in
FR-01	The system shall load applicant records from a CSV file and abort with a clear message if the file is missing.	`data_loader.load_csv`
FR-02	The system shall validate the data: detect duplicate application IDs, count missing values, and flag non-positive income.	`data_loader.validate`
FR-03	The system shall impute missing numeric values with the column median and missing categorical values with the mode.	`data_loader.validate`
FR-04	The system shall compute monthly income, estimated EMI, total EMI, debt-to-income ratio and loan-to-income ratio for every record.	`data_loader.engineer_features`
FR-05	The system shall assign every application a risk score between 0 and 100 using published, inspectable rules.	`risk_engine.score_application`
FR-06	The system shall map each score to a band (LOW / MEDIUM / HIGH) and a suggested action.	`risk_engine` + `config.RISK_BANDS`
FR-07	The system shall return a human-readable reason string listing each penalty and the points it added.	`risk_engine.score_application`
FR-08	The system shall report descriptive statistics and approval rates grouped by employment type.	`eda.print_summary`
FR-09	The system shall generate and save at least four analytical charts as PNG files.	`eda.build_charts`
FR-10	The system shall train a classifier inside a scikit-learn Pipeline so preprocessing is identical at training and prediction time.	`model.build_pipeline`
FR-11	The system shall evaluate the model with accuracy, ROC-AUC, a confusion matrix and a classification report.	`model.train`
FR-12	The system shall persist the trained pipeline to disk and reload it without retraining.	`model.train` / `model.load`
FR-13	The system shall rank and display the model coefficients that most influence the decision.	`model.top_drivers`
FR-14	The system shall process incoming applications one at a time with a configurable delay between them.	`stream_simulator.run`
FR-15	The system shall combine the rule decision and the model probability into one final decision, with the rule engine holding veto power on high risk.	`stream_simulator.combine_decision`
FR-16	The system shall maintain and display rolling KPIs after every processed application.	`stream_simulator.RollingKPI`
FR-17	The system shall write every decision, with its justification, to an audit CSV.	`stream_simulator.run`
FR-18	The system shall write a plain-text summary report of the whole run.	`main.write_summary`
FR-19	The system shall expose command-line flags to skip the delay and to skip chart generation.	`main.main`

4.2 Non-functional requirements

ID	Category	Requirement
NFR-01	Explainability	No decision may be issued without a reason string that a customer could be shown.
NFR-02	Configurability	Every threshold, path and rate lives in `config.py`. No magic numbers anywhere else.
NFR-03	Modularity	Each module runs standalone via its `__main__` block for isolated testing.
NFR-04	Reproducibility	A fixed `random_state` makes every run produce identical results.
NFR-05	Portability	Runs on Windows, macOS, Linux and Google Colab with no code changes; matplotlib uses the headless Agg backend.
NFR-06	Performance	End-to-end run finishes in under 5 seconds with `--fast`; per-record scoring is a few milliseconds.
NFR-07	Auditability	Every run leaves charts, a model file, a decision log and a summary report on disk.
NFR-08	Robustness	Missing files raise a descriptive error; unseen categories are handled by `handle_unknown="ignore"`.
NFR-09	Readability	Every module carries a docstring explaining not just what it does but why.

4.3 Software and hardware requirements

Item	Requirement
Operating system	Windows 10+, macOS 11+, or any modern Linux
Python	3.9 or newer (3.10 / 3.11 recommended)
RAM	2 GB is more than sufficient
Disk	About 60 MB including the Python libraries
Internet	Needed once, only to `pip install` the libraries. The project itself runs fully offline.
Editor	VS Code, PyCharm, Jupyter or plain IDLE
Libraries	pandas, numpy, scikit-learn, matplotlib, joblib (streamlit only for the optional dashboard)

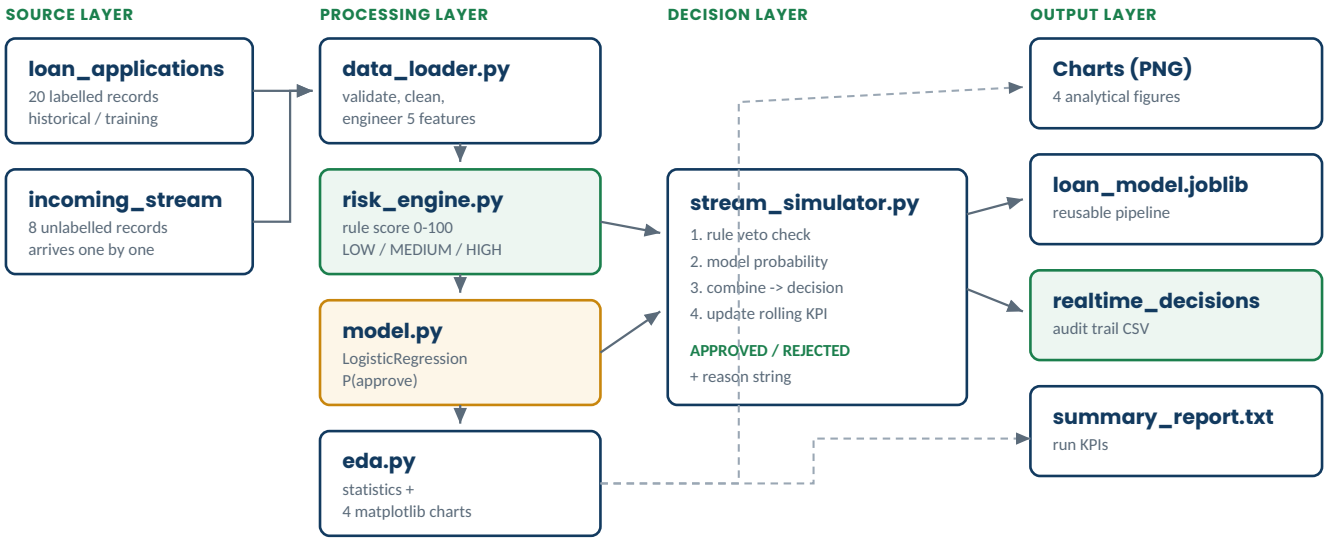
REQUIREMENTS.TXT

```
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
matplotlib>=3.7
joblib>=1.3

# optional, only for app/dashboard.py
# streamlit>=1.30
```

5 System Architecture

The system is a four-layer pipeline. Data moves strictly left to right, and no layer reaches backwards. This is what allows any single stage to be tested, replaced or scaled without touching the others.



Solid arrows = live data flow. Dashed arrows = artefacts written to disk.
Every module is importable on its own, so each stage can be tested in isolation.

Layer	Responsibility	Modules
Source	Provide raw application records, historical and live	`data/*.csv`
Processing	Clean the data, derive lending ratios, compute a risk score, train the model, produce analytics	`data_loader.py`, `risk_engine.py`, `model.py`, `eda.py`
Decision	Arbitrate between the rule engine and the model, produce a final verdict, track live KPIs	`stream_simulator.py`
Output	Persist everything a reviewer or regulator would ask for	`outputs/`

The two-brain design

The rule engine and the model are deliberately independent. The rule engine is *deterministic and explainable* but rigid; the model is *flexible and data-driven* but harder to justify. Running both and letting the rule engine veto the extremes gives you the safety of the first and the accuracy of the second. Banks call this a champion/challenger or policy-overlay architecture.

6 Dataset Description

Two CSV files ship with the project. Both use the same schema, except that the stream file has no `loan_approved` column — that is exactly what the system has to predict.

File	Rows	Has label?	Used for
`data/loan_applications.csv`	20	Yes	Rule calibration, EDA, model training and testing
`data/incoming_stream.csv`	8	No	The simulated live feed of new applications

6.1 Data dictionary

Column	Type	Description	Example
`application_id`	string	Unique reference for the application	LN1001
`applicant_name`	string	Applicant name, used only for display	Arjun Mehta
`age`	int	Age in years	32
`gender`	category	Male / Female	Male
`marital_status`	category	Single / Married / Divorced / Widowed	Married
`employment_type`	category	Salaried / Self-Employed / Unemployed	Salaried
`annual_income`	int	Gross yearly income in rupees	1200000
`credit_score`	int	Bureau score, 300 (worst) to 900 (best)	780
`existing_emi`	int	Monthly instalments already being paid	8000
`loan_amount`	int	Principal requested, in rupees	2500000
`loan_term_months`	int	Repayment period in months	240
`loan_purpose`	category	Home / Personal / Vehicle / Education / Business	Home
`property_area`	category	Urban / Semiurban / Rural	Urban
`previous_defaults`	int	Count of past loans the applicant defaulted on	0
`loan_approved`	target	Y or N — the historical outcome (training file only)	Y

6.2 Sample records

The first six rows of the training file, shown as they appear in the CSV:

DATA/LOAN_APPLICATIONS.CSV (EXTRACT)

```
application_id,applicant_name,age,gender,marital_status,employment_type,annual_income,credit_score,existing_emi,loan_amount,loa
LN1001,Arjun Mehta,32,Male,Married,Salaried,1200000,780,8000,2500000,240,Home,Urban,0,Y
LN1002,Priya Sharma,28,Female,Single,Salaried,850000,720,5000,800000,60,Personal,Urban,0,Y
LN1003,Rahul Verma,45,Male,Married,Self-Employed,600000,610,15000,1500000,120,Business,Semiurban,1,N
LN1004,Sneha Iyer,26,Female,Single,Salaried,480000,690,0,400000,36,Education,Urban,0,Y
LN1005,Vikram Nair,38,Male,Married,Self-Employed,1500000,800,12000,3000000,180,Home,Urban,0,Y
LN1006,Deepa Rao,41,Female,Divorced,Salaried,720000,580,20000,1200000,84,Personal,Rural,2,N
```

6.3 Engineered features

Raw columns are not enough. A rupee income figure means nothing on its own — what matters is income *relative to* debt. These five columns are computed in `engineer_features()` and are the strongest predictors in the whole project.

Feature	Formula	Why lenders care
`monthly_income`	annual_income / 12	EMIs are monthly, so income must be too.
`estimated_emi`	$P \cdot r \cdot (1+r)^n / ((1+r)^n - 1)$	The reducing-balance EMI the applicant would owe if sanctioned. r = monthly rate = 10.5% / 12, n = term in months.
`total_emi`	estimated_emi + existing_emi	Total monthly debt burden after this loan is added.
`dti_ratio`	total_emi / monthly_income	**Debt-to-Income.** The single most important affordability metric. Above roughly 0.50 most lenders decline outright.
`loan_to_income`	loan_amount / annual_income	How many years of gross salary the loan represents. A rough sanity check on loan size.

Verify one by hand

Take LN1001: loan 25,00,000 over 240 months at 10.5% p.a. r = 0.105/12 = 0.00875, so EMI ≈ Rs.24,959. Add the existing EMI of Rs.8,000 and you get Rs.32,959 against a monthly income of Rs.1,00,000 — a DTI of 0.33, which the code prints as 0.330. Being able to reproduce a number by hand is how you know your pipeline is correct.

7 Project Folder Structure

FOLDER TREE

```
BankLoanRiskAnalytics/
|
+-- README.md                              quick start guide
+-- requirements.txt                       library versions
+-- Bank_Loan_Approval_Risk_Analytics.pdf  this handbook
|
+-- data/
|   +-- loan_applications.csv              20 labelled historical records
|   +-- incoming_stream.csv                 8 unlabelled live records
|
+-- src/
|   +-- config.py              all paths, thresholds and business rules
|   +-- data_loader.py         ingestion, validation, feature engineering
|   +-- risk_engine.py         explainable rule-based risk score (0-100)
|   +-- eda.py                 console statistics + 4 matplotlib charts
|   +-- model.py               scikit-learn pipeline, training, evaluation
|   +-- stream_simulator.py    real-time decision loop with rolling KPIs
|   +-- main.py                orchestrator  <-- RUN THIS
|
+-- app/
|   +-- dashboard.py           OPTIONAL Streamlit dashboard
|
+-- sample_outputs/            reference results, for comparison only
|   +-- console_output.txt     the full console log of a reference run
|   +-- ... the same 7 files listed below
|
+-- outputs/                   EMPTY until you run the project
    +-- 01_approval_distribution.png
    +-- 02_credit_score_boxplot.png
    +-- 03_income_vs_loan.png
    +-- 04_risk_bands.png
    +-- loan_model.joblib
    +-- realtime_decisions.csv
    +-- summary_report.txt
```

Why src/ has no package prefix

The modules import each other plainly (`import config`) and you run `python main.py` from inside `src/` . This keeps imports readable for beginners. In a production repository you would add an `__init__.py` , turn `src` into a package and import as `from src import config` .

8 Methodology: The Five Stages

Running `python main.py` executes these five stages in order. Each is printed with a banner so you can follow along.

Stage	Name	Module	What happens
Stage 1	Data ingestion and feature engineering	`data_loader.py`	Read the CSV, check for duplicates and missing values, impute what is missing, then derive monthly income, EMI, total EMI, DTI and loan-to-income. Output: a clean DataFrame with 24 columns.
Stage 2	Rule-based risk scoring	`risk_engine.py`	Apply five families of penalty rules (credit history, affordability, loan size, past defaults, income stability) to produce a 0–100 score, a band and a reason string for every record.
Stage 3	Exploratory data analysis	`eda.py`	Print approval rates, group averages and correlations; save four charts. This is where you form hypotheses about what drives approval before you let a model near the data.
Stage 4	Model training and evaluation	`model.py`	Build a Pipeline (scale numerics, one-hot encode categoricals, fit Logistic Regression), split 70/30 with stratification, evaluate, rank coefficients, and persist the fitted pipeline to disk.
Stage 5	Real-time decisioning	`stream_simulator.py`	Consume the incoming stream one record at a time. For each: engineer features, score with rules, predict with the model, arbitrate, update rolling KPIs, print a live line, append to the audit log.

8.1 Data flow in one sentence per hop

1. CSV file → `pandas.DataFrame` (15 columns).
2. DataFrame → validated DataFrame (no duplicates, no nulls).
3. Validated DataFrame → enriched DataFrame (20 columns after 5 engineered features).
4. Enriched DataFrame → scored DataFrame (24 columns after risk score, band, decision, reasons).
5. Scored DataFrame → feature matrix X and label vector y → fitted Pipeline → `loan_model.joblib`.
6. Stream row → enriched Series → rule result + model probability → final decision → KPI update → log row.

9 The Risk Scoring Rulebook

> Score 0 means a flawless borrower. Score 100 means the worst possible profile. Penalties are additive and the total is capped at 100.

Every threshold below lives in `config.py`, so you can tighten or loosen the bank's appetite for risk by editing one file and re-running — no other code changes.

9.1 Rule 1 — Credit history

Credit score	Penalty	Interpretation
750 and above	0	Excellent bureau record
700 – 749	+8	Good
650 – 699	+18	Fair, watch closely
600 – 649	+30	Poor
Below 600	+45	Very poor, likely prior distress

9.2 Rule 2 — Affordability (debt-to-income)

DTI ratio	Penalty	Interpretation
Up to 0.30	0	Comfortable repayment capacity
0.31 – 0.40	+10	Acceptable
0.41 – 0.50	+22	Stretched
Above 0.50	+35	Over-leveraged; more than half of income already committed

9.3 Rule 3 — Loan size relative to income

Loan-to-income	Penalty	Interpretation
Up to 1.5x	0	Modest ask
1.6x – 2.5x	+6	Normal for secured lending
2.6x – 3.5x	+12	Large relative to earnings
Above 3.5x	+20	Disproportionate to income

9.4 Rules 4 and 5 — Past behaviour and income stability

Condition	Penalty	Interpretation
Each previous default	+12 per default	The strongest single predictor of future default
Employment = Unemployed	+15	No verifiable income stream
Employment = Self-Employed	+5	Income is real but variable and harder to document

9.5 Bands and actions

Score range	Band	Suggested action	Meaning
0 – 25	LOW	AUTO-APPROVE	Straight-through processing, no human needed
26 – 45	MEDIUM	MANUAL REVIEW	A credit officer must look at the file
46 – 100	HIGH	AUTO-REJECT	Rejected by policy; the model cannot overrule

9.6 A worked example

Application LN2005 (Naveen Prasad): income Rs.3,00,000 a year, credit score 540, an existing EMI of Rs.17,000, requesting Rs.12,00,000 over 120 months, self-employed, two previous defaults.

Rule	Value	Penalty
Credit history	540 → below 600	+45
Affordability	DTI 1.33 → above 0.50	+35
Loan size	4.0x income → above 3.5x	+20
Previous defaults	2 × 12	+24
Income stability	Self-Employed	+5
Raw total		**129**
Capped score		**100 → HIGH → AUTO-REJECT**

This is what explainability looks like

The applicant can be told exactly why: *credit score 540 (+45); DTI 133% (+35); loan is 4.0x income (+20); 2 previous default(s) (+24); self-employed income (+5)*. A neural network could reach the same conclusion but could not produce that sentence — which is why regulated lenders keep a rule layer.

10 Code Walkthrough

> Seven modules, about 700 lines in total. Each one is printed here in full and then explained. Read them in this order — each builds on the one before it.

Order	Module	Lines	One-line job
1	`config.py`	~95	Every path, threshold and business rule in one place
2	`data_loader.py`	~110	Read, validate, clean and enrich the data
3	`risk_engine.py`	~105	Turn a record into an explainable 0–100 risk score
4	`eda.py`	~110	Describe the data in numbers and pictures
5	`model.py`	~130	Train, evaluate, interpret and persist the classifier
6	`stream_simulator.py`	~140	The real-time decision loop
7	`main.py`	~110	Wire the five stages together

10.1 config.py — the single source of truth

Beginners scatter file paths and magic numbers through their code. Then a threshold changes and they have to hunt through five files. This module exists so that never happens: **if it is a number a business person could argue about, it lives here.**

SRC/CONFIG.PY

```
"""
config.py
---------
Single place for every path, constant and business rule threshold used by the
project. Nothing else in the codebase should hard-code a number or a file path.
"""

from pathlib import Path

# ---------------------------------------------------------------------------
# 1. FOLDER LAYOUT
# ---------------------------------------------------------------------------
# __file__ -> .../BankLoanRiskAnalytics/src/config.py
# .parent  -> .../src        .parent.parent -> project root
BASE_DIR = Path(__file__).resolve().parent.parent

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "outputs"

TRAINING_CSV = DATA_DIR / "loan_applications.csv"
STREAM_CSV = DATA_DIR / "incoming_stream.csv"

MODEL_FILE = OUTPUT_DIR / "loan_model.joblib"
DECISION_LOG = OUTPUT_DIR / "realtime_decisions.csv"
SUMMARY_REPORT = OUTPUT_DIR / "summary_report.txt"

# Create the outputs folder on import so no module has to worry about it.
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------------------------
# 2. COLUMN GROUPS
# ---------------------------------------------------------------------------
TARGET = "loan_approved"
```

```
# Raw numeric columns coming straight from the CSV
NUMERIC_RAW = [
    "age",
    "annual_income",
    "credit_score",
    "existing_emi",
    "loan_amount",
    "loan_term_months",
    "previous_defaults",
]

# Columns created by feature engineering (see data_loader.engineer_features)
NUMERIC_ENGINEERED = [
    "monthly_income",
    "estimated_emi",
    "total_emi",
    "dti_ratio",
    "loan_to_income",
]

CATEGORICAL = [
    "gender",
    "marital_status",
    "employment_type",
    "loan_purpose",
    "property_area",
]

FEATURES = NUMERIC_RAW + NUMERIC_ENGINEERED + CATEGORICAL

# ---------------------------------------------------------------------------
# 3. BUSINESS RULES (used by risk_engine.py)
# ---------------------------------------------------------------------------
ANNUAL_INTEREST_RATE = 0.105          # 10.5 % p.a. flat assumption for EMI maths

CREDIT_SCORE_BANDS = [                # (minimum score, penalty points)
    (750, 0),
    (700, 8),
    (650, 18),
    (600, 30),
    (0, 45),
]

DTI_BANDS = [                         # (maximum DTI, penalty points)
    (0.30, 0),
    (0.40, 10),
    (0.50, 22),
    (1.00, 35),
]

LTI_BANDS = [                         # (maximum loan-to-income, penalty points)
    (1.5, 0),
    (2.5, 6),
    (3.5, 12),
    (99.0, 20),
]

DEFAULT_PENALTY = 12                  # points per previous default
UNEMPLOYED_PENALTY = 15
SELF_EMPLOYED_PENALTY = 5

RISK_BANDS = [                        # (maximum score, label, decision hint)
    (25, "LOW", "AUTO-APPROVE"),
    (45, "MEDIUM", "MANUAL REVIEW"),
    (100, "HIGH", "AUTO-REJECT"),
]
```

```
# ---------------------------------------------------------------------------
# 4. MODEL / STREAM SETTINGS
# ---------------------------------------------------------------------------
TEST_SIZE = 0.30
RANDOM_STATE = 42
APPROVAL_THRESHOLD = 0.50            # probability above which the model says "Y"
STREAM_DELAY_SECONDS = 1.0           # pause between two simulated applications
```

Construct	What it does and why it matters
`BASE_DIR = Path(__file__).resolve().parent.parent`	Builds every other path relative to the project root instead of the current working directory. This is why the project runs correctly whether you launch it from `src/`, from the project root, or from an IDE.
`OUTPUT_DIR.mkdir(parents=True, exist_ok=True)`	Runs at import time, so no other module ever has to check whether `outputs/` exists. `exist_ok=True` makes it safe to run repeatedly.
`NUMERIC_RAW / NUMERIC_ENGINEERED / CATEGORICAL`	Column groups, not loose strings. `model.py` feeds these lists straight into the `ColumnTransformer`, so adding a new feature is a one-line change here.
`CREDIT_SCORE_BANDS` etc.	Rules expressed as data (a list of tuples) rather than as `if/elif` chains. The rule engine walks the list, which means changing policy never means changing logic.
`RANDOM_STATE = 42`	Fixes the train/test shuffle so your results match this handbook exactly. Remove it and your accuracy will change on every run.

10.2 data_loader.py — ingestion and feature engineering

Three responsibilities: get the data in, prove it is usable, and add the columns that make it predictive. In industry this is called the ETL or feature-pipeline layer, and it is usually 70% of the work on a real project.

SRC/DATA_LOADER.PY

```
"""
data_loader.py
--------------
Stage 1 of the pipeline: read the CSV, validate it, clean it and derive the
extra columns that both the rule engine and the ML model depend on.
"""

import pandas as pd

import config


def load_csv(path):
    """Read a CSV file into a DataFrame and fail loudly if it is missing."""
    if not path.exists():
        raise FileNotFoundError(f"Required data file not found: {path}")
    df = pd.read_csv(path)
    print(f"[data_loader] Loaded {len(df)} rows x {len(df.columns)} columns from {path.name}")
    return df


def validate(df, expect_target=True):
    """Basic data-quality gate. Returns a dict describing what was found."""
    report = {
        "rows": len(df),
        "duplicate_ids": int(df["application_id"].duplicated().sum()),
        "missing_values": int(df.isna().sum().sum()),
        "negative_income": int((df["annual_income"] <= 0).sum()),
    }

    if expect_target and config.TARGET not in df.columns:
```

```
        raise ValueError(f"Target column '{config.TARGET}' is missing")

    if report["duplicate_ids"]:
        df = df.drop_duplicates(subset="application_id", keep="first")

    # Numeric columns: fill any gap with the column median (robust to outliers)
    for col in config.NUMERIC_RAW:
        if df[col].isna().any():
            df[col] = df[col].fillna(df[col].median())

    # Categorical columns: fill gaps with the most frequent value
    for col in config.CATEGORICAL:
        if df[col].isna().any():
            df[col] = df[col].fillna(df[col].mode()[0])

    print(f"[data_loader] Validation -> {report}")
    return df, report


def _emi(principal, annual_rate, months):
    """
    Standard reducing-balance EMI formula.

        EMI = P * r * (1+r)^n / ((1+r)^n - 1)

    P = principal, r = monthly interest rate, n = number of months.
    """
    r = annual_rate / 12
    growth = (1 + r) ** months
    return principal * r * growth / (growth - 1)


def engineer_features(df):
    """Create the derived columns listed in config.NUMERIC_ENGINEERED."""
    df = df.copy()

    # Lenders think in months, the CSV stores yearly income.
    df["monthly_income"] = df["annual_income"] / 12

    # EMI the applicant would pay if this loan is sanctioned.
    df["estimated_emi"] = _emi(
        df["loan_amount"], config.ANNUAL_INTEREST_RATE, df["loan_term_months"]
    ).round(0)

    # Total monthly obligation = new EMI + EMIs already running.
    df["total_emi"] = df["estimated_emi"] + df["existing_emi"]

    # Debt-to-Income: the single most important affordability metric.
    df["dti_ratio"] = (df["total_emi"] / df["monthly_income"]).round(3)

    # Loan-to-Income: how many years of gross salary the loan represents.
    df["loan_to_income"] = (df["loan_amount"] / df["annual_income"]).round(3)

    print(f"[data_loader] Engineered {len(config.NUMERIC_ENGINEERED)} new features")
    return df


def get_training_data():
    """Convenience wrapper used by main.py -> returns a model-ready DataFrame."""
    df = load_csv(config.TRAINING_CSV)
    df, _ = validate(df, expect_target=True)
    return engineer_features(df)


def get_stream_data():
    """Same as above but for the unlabelled incoming applications."""
    df = load_csv(config.STREAM_CSV)
```

```
    df, _ = validate(df, expect_target=False)
    return engineer_features(df)


if __name__ == "__main__":
    data = get_training_data()
    print(data[["application_id", "dti_ratio", "loan_to_income", "estimated_emi"]].head())
```

Function	Explanation
`load_csv(path)`	Checks existence first and raises `FileNotFoundError` with the offending path. Failing loudly and early beats a confusing `KeyError` forty lines later.
`validate(df, expect_target)`	Builds a small report dict (row count, duplicate IDs, missing values, non-positive income), removes duplicate application IDs, then imputes: **median** for numerics because it resists outliers (one crorepati would drag a mean upward), **mode** for categoricals because there is no such thing as an average category. `expect_target` lets the same function serve both the labelled training file and the unlabelled stream.
`_emi(principal, rate, months)`	The reducing-balance EMI formula. The leading underscore is a Python convention meaning *internal helper — not part of the public interface*. Because it is written with pandas arithmetic, passing a whole Series computes every EMI at once, with no Python loop.
`engineer_features(df)`	Starts with `df.copy()` so the caller's DataFrame is never mutated behind their back — a habit that prevents a whole class of bugs. Then adds the five derived columns described in Section 6.3.
`get_training_data()` / `get_stream_data()`	Thin wrappers that chain load → validate → engineer. `main.py` calls one line instead of three, and the stream is guaranteed to receive exactly the same treatment as the training data.

The most common real-world bug this prevents

If the stream were enriched differently from the training data — say DTI computed without the existing EMI — the model would silently produce garbage. Routing both through the same functions is the cheapest insurance against **training/serving skew**.

10.3 risk_engine.py — explainable scoring

This is the module a regulator would read. It contains no machine learning at all: just arithmetic that anyone can follow, producing a score, a band, an action and a sentence explaining itself.

SRC/RISK_ENGINE.PY

```
"""
risk_engine.py
--------------
Stage 2: a transparent, rule-based credit risk score.

Why a rule engine when we also train an ML model?
Banks are regulated. A model can output "reject" but the officer must still be
able to tell the customer *why*. The rule engine gives that explanation, and it
also works on day one, before enough history exists to train anything.

Score = 0 (perfect borrower) ... 100 (worst borrower). Penalties are additive.
"""

import config


def _band_penalty(value, bands, higher_is_better):
    """
    Walk a list of (threshold, penalty) tuples and return the first match.

    higher_is_better=True  -> bands are (minimum, penalty), e.g. credit score
    higher_is_better=False -> bands are (maximum, penalty), e.g. DTI ratio
```

```
    """
    for threshold, penalty in bands:
        if higher_is_better and value >= threshold:
            return penalty
        if not higher_is_better and value <= threshold:
            return penalty
    return bands[-1][1]


def score_application(row):
    """
    Score one application (a pandas Series or a plain dict).

    Returns a dict with the total score, the risk band, the suggested decision
    and a human-readable breakdown of every penalty that was applied.
    """
    reasons = []
    score = 0

    # --- 1. Credit history ------------------------------------------------
    p = _band_penalty(row["credit_score"], config.CREDIT_SCORE_BANDS, True)
    if p:
        reasons.append(f"credit score {int(row['credit_score'])} (+{p})")
    score += p

    # --- 2. Affordability (debt-to-income) --------------------------------
    p = _band_penalty(row["dti_ratio"], config.DTI_BANDS, False)
    if p:
        reasons.append(f"DTI {row['dti_ratio']:.0%} (+{p})")
    score += p

    # --- 3. Loan size relative to income ----------------------------------
    p = _band_penalty(row["loan_to_income"], config.LTI_BANDS, False)
    if p:
        reasons.append(f"loan is {row['loan_to_income']:.1f}x income (+{p})")
    score += p

    # --- 4. Past behaviour ------------------------------------------------
    defaults = int(row["previous_defaults"])
    if defaults:
        p = defaults * config.DEFAULT_PENALTY
        reasons.append(f"{defaults} previous default(s) (+{p})")
        score += p

    # --- 5. Income stability ----------------------------------------------
    employment = row["employment_type"]
    if employment == "Unemployed":
        reasons.append(f"unemployed (+{config.UNEMPLOYED_PENALTY})")
        score += config.UNEMPLOYED_PENALTY
    elif employment == "Self-Employed":
        reasons.append(f"self-employed income (+{config.SELF_EMPLOYED_PENALTY})")
        score += config.SELF_EMPLOYED_PENALTY

    score = min(score, 100)          # cap so the scale stays 0-100

    # --- 6. Translate the number into a band + action ---------------------
    band, action = config.RISK_BANDS[-1][1], config.RISK_BANDS[-1][2]
    for limit, label, hint in config.RISK_BANDS:
        if score <= limit:
            band, action = label, hint
            break

    return {
        "risk_score": score,
        "risk_band": band,
        "rule_decision": action,
        "risk_reasons": "; ".join(reasons) if reasons else "no risk flags",
```

```
    }


def score_dataframe(df):
    """Vectorised-by-loop version: adds four risk columns to a whole DataFrame."""
    scored = df.apply(score_application, axis=1, result_type="expand")
    out = df.join(scored)
    counts = out["risk_band"].value_counts().to_dict()
    print(f"[risk_engine] Scored {len(out)} applications -> {counts}")
    return out


if __name__ == "__main__":
    import data_loader

    demo = score_dataframe(data_loader.get_training_data())
    print(demo[["application_id", "risk_score", "risk_band", "rule_decision"]].head())
```

Element	Explanation
`_band_penalty(value, bands, higher_is_better)`	One generic function handles both directions of comparison. For credit score, higher is better, so it returns the first band whose *minimum* the value clears. For DTI and LTI, lower is better, so it returns the first band whose *maximum* the value stays under. Because the band lists are ordered, the first match is always the right one.
`reasons` list	Every penalty appends a short phrase such as `DTI 65% (+35)`. Joined with semicolons at the end, this becomes the justification shown to the customer. Building it *as the score is computed* guarantees the explanation can never drift out of sync with the number.
`score = min(score, 100)`	Caps the scale. Without it a truly terrible applicant could score 129 (as LN2005 does before capping) and break any chart with a 0–100 axis.
Band lookup loop	Walks `config.RISK_BANDS` and breaks at the first threshold the score fits under. The default before the loop is the worst band, so an unexpected value fails safe towards rejection rather than approval.
`df.apply(..., result_type="expand")`	The function returns a dict per row; `expand` turns those dicts into four proper DataFrame columns in one pass. This is the idiomatic pandas way to apply a multi-output function.
`if __name__ == "__main__"`	Lets you run `python risk_engine.py` to test the rules alone. This block never executes when the module is imported.

10.4 eda.py — understanding the data before modelling

Never train a model on data you have not looked at. This module answers: how imbalanced is the target, which variables separate the two classes, and are there any correlations strong enough to be suspicious?

SRC/EDA.PY

```
"""
eda.py
------
Stage 3: exploratory data analysis. Prints the numbers a credit manager would
ask for and saves four PNG charts into outputs/.

matplotlib only (no seaborn) so the project runs with the smallest possible
dependency list.
"""

import matplotlib

matplotlib.use("Agg")            # headless backend: works on servers / Colab
import matplotlib.pyplot as plt

import config
```

```
PALETTE = {"approve": "#1b7f4d", "reject": "#b3261e", "accent": "#1f4e79", "grey": "#8a8a8a"}


def print_summary(df):
    """Console EDA: shape, approval rate, group averages."""
    print("\n" + "=" * 68)
    print("EXPLORATORY DATA ANALYSIS")
    print("=" * 68)

    print(f"Records            : {len(df)}")
    print(f"Columns            : {len(df.columns)}")

    approved = (df[config.TARGET] == "Y").sum()
    print(f"Approved / Rejected: {approved} / {len(df) - approved}")
    print(f"Approval rate      : {approved / len(df):.1%}")

    print("\n-- Averages by approval status --")
    grouped = df.groupby(config.TARGET)[
        ["credit_score", "annual_income", "dti_ratio", "loan_to_income", "previous_defaults"]
    ].mean().round(2)
    print(grouped.to_string())

    print("\n-- Approval rate by employment type --")
    emp = df.groupby("employment_type")[config.TARGET].apply(
        lambda s: (s == "Y").mean()
    ).sort_values(ascending=False)
    for name, rate in emp.items():
        print(f"  {name:<15} {rate:.0%}")

    print("\n-- Correlation of numeric drivers with approval --")
    numeric = df[["credit_score", "annual_income", "dti_ratio", "loan_to_income",
                  "previous_defaults", "risk_score"]].copy()
    numeric["approved_flag"] = (df[config.TARGET] == "Y").astype(int)
    corr = numeric.corr(numeric_only=True)["approved_flag"].drop("approved_flag").round(3)
    print(corr.sort_values(ascending=False).to_string())


def _save(fig, filename):
    path = config.OUTPUT_DIR / filename
    fig.tight_layout()
    fig.savefig(path, dpi=130)
    plt.close(fig)
    print(f"[eda] saved {path.name}")


def build_charts(df):
    """Four charts that together tell the whole data story."""

    # 1. Approval split ------------------------------------------------------
    fig, ax = plt.subplots(figsize=(5, 4))
    counts = df[config.TARGET].value_counts()
    ax.bar(["Approved" if k == "Y" else "Rejected" for k in counts.index], counts.values,
           color=[PALETTE["approve"] if k == "Y" else PALETTE["reject"] for k in counts.index])
    for i, v in enumerate(counts.values):
        ax.text(i, v + 0.15, str(v), ha="center", fontweight="bold")
    ax.set_title("Loan approval distribution")
    ax.set_ylabel("Applications")
    _save(fig, "01_approval_distribution.png")

    # 2. Credit score vs outcome ---------------------------------------------
    fig, ax = plt.subplots(figsize=(5, 4))
    data = [df.loc[df[config.TARGET] == "N", "credit_score"],
            df.loc[df[config.TARGET] == "Y", "credit_score"]]
    bp = ax.boxplot(data, tick_labels=["Rejected", "Approved"], patch_artist=True)
    for patch, colour in zip(bp["boxes"], [PALETTE["reject"], PALETTE["approve"]]):
        patch.set_facecolor(colour)
```

```
        patch.set_alpha(0.55)
    ax.set_title("Credit score by outcome")
    ax.set_ylabel("CIBIL score")
    _save(fig, "02_credit_score_boxplot.png")

    # 3. Income vs loan amount ------------------------------------------------
    fig, ax = plt.subplots(figsize=(5.5, 4))
    for label, colour, name in [("Y", PALETTE["approve"], "Approved"),
                                ("N", PALETTE["reject"], "Rejected")]:
        sub = df[df[config.TARGET] == label]
        ax.scatter(sub["annual_income"] / 1e5, sub["loan_amount"] / 1e5,
                   s=sub["risk_score"] * 4 + 20, c=colour, alpha=0.65,
                   edgecolors="white", label=name)
    ax.set_xlabel("Annual income (lakh INR)")
    ax.set_ylabel("Loan amount (lakh INR)")
    ax.set_title("Income vs loan size (bubble = risk score)")
    ax.legend()
    _save(fig, "03_income_vs_loan.png")

    # 4. Risk band distribution -----------------------------------------------
    fig, ax = plt.subplots(figsize=(5, 4))
    order = ["LOW", "MEDIUM", "HIGH"]
    values = [int((df["risk_band"] == b).sum()) for b in order]
    ax.bar(order, values, color=[PALETTE["approve"], "#c8860d", PALETTE["reject"]])
    for i, v in enumerate(values):
        ax.text(i, v + 0.15, str(v), ha="center", fontweight="bold")
    ax.set_title("Applications per risk band (rule engine)")
    ax.set_ylabel("Applications")
    _save(fig, "04_risk_bands.png")


def run(df):
    print_summary(df)
    build_charts(df)


if __name__ == "__main__":
    import data_loader
    import risk_engine

    run(risk_engine.score_dataframe(data_loader.get_training_data()))
```

Element	Explanation
`matplotlib.use("Agg")`	Selects a non-interactive backend **before** importing pyplot. Without it, matplotlib tries to open a window and crashes on servers, in Docker and inside CI pipelines.
`df.groupby(TARGET)[cols].mean()`	The single most informative line in the whole EDA. Approved applicants average a credit score of 740 against 608 for rejected ones, and a DTI of 0.35 against 0.72 — the story of the dataset in two numbers.
`.apply(lambda s: (s == "Y").mean())`	A boolean Series averages to a proportion, so this computes the approval *rate* per employment type in one expression.
Correlation with `approved_flag`	Converts Y/N to 1/0 and correlates every numeric driver against it. Sign matters: credit score is positive, DTI and defaults are negative, exactly as domain knowledge predicts.
`_save(fig, filename)`	Private helper that applies `tight_layout()`, writes at 130 DPI and — crucially — calls `plt.close(fig)`. Forgetting to close figures inside a loop is the classic matplotlib memory leak.
Bubble scatter	`s=sub["risk_score"] * 4 + 20` encodes a third variable in the marker size, so one chart shows income, loan size, outcome and risk simultaneously.

10.5 model.py — the machine learning layer

The goal is not the highest possible accuracy. It is a model that is **correct, reproducible and explainable**, wrapped so it can be reused in the streaming layer without any risk of preprocessing drift.

SRC/MODEL.PY

```
"""
model.py
--------
Stage 4: train a Logistic Regression classifier that predicts whether a loan
will be approved, and expose a predict() helper the real-time engine can call.

Logistic Regression is chosen deliberately:
  * it outputs a calibrated probability, not just a label
  * its coefficients are readable, which matters for credit regulation
  * it is stable on very small datasets, unlike deep trees
"""

import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

import config


def build_pipeline():
    """
    A Pipeline bundles preprocessing + estimator into one object, so the exact
    same transformations are applied at training time and at prediction time.
    This is what prevents 'training/serving skew'.
    """
    numeric = config.NUMERIC_RAW + config.NUMERIC_ENGINEERED

    preprocess = ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), numeric),
            ("cat", OneHotEncoder(handle_unknown="ignore"), config.CATEGORICAL),
        ]
    )

    return Pipeline(
        steps=[
            ("preprocess", preprocess),
            ("classifier", LogisticRegression(max_iter=1000,
                                              class_weight="balanced",
                                              random_state=config.RANDOM_STATE)),
        ]
    )


def train(df):
    """Split, fit, evaluate and persist. Returns (pipeline, metrics dict)."""
    X = df[config.FEATURES]
    y = (df[config.TARGET] == "Y").astype(int)      # Y/N -> 1/0

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=config.TEST_SIZE,
```

```
        random_state=config.RANDOM_STATE,
        stratify=y,                                  # keep class balance in both sets
    )

    pipe = build_pipeline()
    pipe.fit(X_train, y_train)

    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1]

    metrics = {
        "train_rows": len(X_train),
        "test_rows": len(X_test),
        "accuracy": round(accuracy_score(y_test, y_pred), 3),
        "roc_auc": round(roc_auc_score(y_test, y_prob), 3) if y_test.nunique() > 1 else None,
        "confusion_matrix": confusion_matrix(y_test, y_pred).tolist(),
    }

    print("\n" + "=" * 68)
    print("MODEL TRAINING")
    print("=" * 68)
    print(f"Train rows : {metrics['train_rows']}   Test rows : {metrics['test_rows']}")
    print(f"Accuracy   : {metrics['accuracy']}")
    print(f"ROC-AUC    : {metrics['roc_auc']}")
    print("Confusion matrix [[TN FP] [FN TP]]:")
    print(np.array(metrics["confusion_matrix"]))
    print("\nClassification report:")
    print(classification_report(y_test, y_pred, target_names=["Rejected", "Approved"],
                                zero_division=0))

    joblib.dump(pipe, config.MODEL_FILE)
    print(f"[model] pipeline saved to {config.MODEL_FILE.name}")

    return pipe, metrics


def top_drivers(pipe, n=8):
    """Read the fitted coefficients back out and rank them by absolute weight."""
    names = pipe.named_steps["preprocess"].get_feature_names_out()
    coefs = pipe.named_steps["classifier"].coef_[0]
    table = (pd.DataFrame({"feature": names, "coefficient": coefs.round(3)})
             .assign(strength=lambda d: d["coefficient"].abs())
             .sort_values("strength", ascending=False)
             .head(n)
             .drop(columns="strength")
             .reset_index(drop=True))
    print("\nTop model drivers (positive = pushes towards approval):")
    print(table.to_string(index=False))
    return table


def load():
    """Load the persisted pipeline; train first if it does not exist yet."""
    if not config.MODEL_FILE.exists():
        raise FileNotFoundError("Model not trained yet. Run main.py first.")
    return joblib.load(config.MODEL_FILE)


def predict_one(pipe, row):
    """
    Score a single application.
    `row` is a pandas Series; we wrap it in a one-row DataFrame because
    scikit-learn always expects 2-D input.
    """
    frame = row.to_frame().T[config.FEATURES]
    probability = float(pipe.predict_proba(frame)[0, 1])
    decision = "APPROVE" if probability >= config.APPROVAL_THRESHOLD else "REJECT"
```

```
    return probability, decision


if __name__ == "__main__":
    import data_loader
    import risk_engine

    frame = risk_engine.score_dataframe(data_loader.get_training_data())
    model, _ = train(frame)
    top_drivers(model)
```

Element	Explanation
`ColumnTransformer`	Applies different transformations to different column groups in one object: `StandardScaler` on the twelve numeric columns, `OneHotEncoder` on the five categorical ones.
`StandardScaler`	Rescales each numeric column to mean 0 and standard deviation 1. Without it, `annual_income` (about 10^6) would dwarf `dti_ratio` (about 0.4) and the regression coefficients would be meaningless to compare.
`OneHotEncoder(handle_unknown="ignore")`	Turns `employment_type = Salaried` into a set of 0/1 columns. `handle_unknown="ignore"` is what stops the stream from crashing if a brand-new category ever arrives — it encodes as all zeros instead of raising.
`Pipeline([...])`	Chains preprocessing and estimator into one fitted object. Calling `pipe.predict()` automatically re-applies the *exact* scaling and encoding learned during training. This one object is what gets saved to disk.
`class_weight="balanced"`	The dataset is 12 approvals to 8 rejections. This tells the regression to weight the minority class more heavily so it does not simply learn to say 'approve' most of the time.
`stratify=y`	Preserves the 60/40 class ratio inside both the training and test split. With only 20 rows, an unstratified split could easily produce a test set with zero rejections, making evaluation impossible.
`predict_proba(X)[:, 1]`	Column 1 is the probability of class 1 (approved). A probability is far more useful than a hard label: it lets the business move the cut-off up or down with the economic cycle.
`roc_auc_score`	Measures ranking quality across *all* thresholds. 0.5 is a coin flip, 1.0 is perfect separation. It is threshold-independent, which makes it the more honest metric when classes are imbalanced.
`joblib.dump(pipe, ...)`	Serialises the fitted pipeline. joblib beats plain pickle here because it compresses the large NumPy arrays inside scikit-learn objects efficiently.
`top_drivers()`	Recovers the transformed feature names with `get_feature_names_out()` and pairs them with the fitted coefficients. Positive means the feature pushes towards approval, negative means towards rejection, and the magnitude ranks importance (valid here precisely because everything was scaled).
`predict_one(pipe, row)`	`row.to_frame().T` converts a single Series into a one-row DataFrame, because scikit-learn always expects 2-D input. Selecting `[config.FEATURES]` guarantees the column order matches training.

10.6 stream_simulator.py — the real-time engine

This is the heart of the project. Everything before it was preparation; this module is what actually behaves like a live system.

SRC/STREAM_SIMULATOR.PY

```
"""
stream_simulator.py
-------------------
Stage 5: the "real-time" part of the project.

A real bank receives applications one at a time from a queue (Kafka, RabbitMQ,
an API endpoint). We reproduce that behaviour by reading incoming_stream.csv
row by row with a delay, and for every arriving record we:

    1. engineer the same features used at training time
    2. run the rule engine  -> explainable risk score + band
    3. run the ML model     -> approval probability
    4. combine both into a final decision
    5. update rolling KPIs and append the result to a decision log

Because the loop is event driven, swapping the CSV reader for a real Kafka
consumer would not change any other line of this file.
"""

import time
from datetime import datetime

import pandas as pd

import config
import data_loader
import model as model_module
import risk_engine


def combine_decision(rule_action, probability):
    """
    Champion/challenger logic used by most retail banks.

    * The rule engine has veto power on the extremes (regulatory safety net).
    * In the grey zone the model's probability decides.
    """
    if rule_action == "AUTO-REJECT":
        return "REJECTED", "rule veto: high risk band"
    if rule_action == "AUTO-APPROVE" and probability >= 0.40:
        return "APPROVED", "low risk + model agrees"
    if probability >= config.APPROVAL_THRESHOLD:
        return "APPROVED", f"model probability {probability:.2f}"
    return "REJECTED", f"model probability {probability:.2f}"


class RollingKPI:
    """Keeps the live counters that a monitoring dashboard would show."""

    def __init__(self):
        self.total = 0
        self.approved = 0
        self.risk_sum = 0
        self.high_risk = 0
        self.exposure = 0          # rupees sanctioned

    def update(self, decision, risk_score, band, loan_amount):
        self.total += 1
        self.risk_sum += risk_score
        if decision == "APPROVED":
            self.approved += 1
            self.exposure += loan_amount
        if band == "HIGH":
            self.high_risk += 1

    def snapshot(self):
```

```
        return {
            "processed": self.total,
            "approval_rate": self.approved / self.total if self.total else 0,
            "avg_risk": self.risk_sum / self.total if self.total else 0,
            "high_risk": self.high_risk,
            "exposure": self.exposure,
        }


def run(pipe=None, delay=None):
    """Process the whole incoming stream and return the decision log."""
    pipe = pipe or model_module.load()
    delay = config.STREAM_DELAY_SECONDS if delay is None else delay

    stream = data_loader.get_stream_data()
    kpi = RollingKPI()
    log = []

    print("\n" + "=" * 68)
    print("REAL-TIME DECISION STREAM")
    print("=" * 68)

    for _, row in stream.iterrows():
        time.sleep(delay)                                    # simulate arrival gap
        arrived = datetime.now().strftime("%H:%M:%S")

        risk = risk_engine.score_application(row)            # explainable layer
        enriched = pd.concat([row, pd.Series(risk)])
        probability, _ = model_module.predict_one(pipe, enriched)   # statistical layer
        decision, why = combine_decision(risk["rule_decision"], probability)

        kpi.update(decision, risk["risk_score"], risk["risk_band"], row["loan_amount"])
        snap = kpi.snapshot()

        print(f"\n[{arrived}] {row['application_id']} | {row['applicant_name']}")
        print(f"    income Rs.{row['annual_income']:,}/yr | loan Rs.{row['loan_amount']:,}"
              f" | CIBIL {row['credit_score']} | DTI {row['dti_ratio']:.0%}")
        print(f"    risk {risk['risk_score']}/100 ({risk['risk_band']})"
              f" | model P(approve)={probability:.2f}")
        print(f"    DECISION: {decision}  <- {why}")
        print(f"    reasons : {risk['risk_reasons']}")
        print(f"    LIVE KPI: processed={snap['processed']}"
              f" approval={snap['approval_rate']:.0%}"
              f" avg-risk={snap['avg_risk']:.1f}"
              f" high-risk={snap['high_risk']}"
              f" exposure=Rs.{snap['exposure']:,}")

        log.append({
            "timestamp": arrived,
            "application_id": row["application_id"],
            "applicant_name": row["applicant_name"],
            "credit_score": row["credit_score"],
            "dti_ratio": row["dti_ratio"],
            "risk_score": risk["risk_score"],
            "risk_band": risk["risk_band"],
            "model_probability": round(probability, 3),
            "final_decision": decision,
            "justification": why,
            "risk_reasons": risk["risk_reasons"],
        })

    log_df = pd.DataFrame(log)
    log_df.to_csv(config.DECISION_LOG, index=False)
    print(f"\n[stream] decision log written to {config.DECISION_LOG.name}")

    final = kpi.snapshot()
    print("\n--- SESSION SUMMARY ---")
```

```
    print(f"Applications processed : {final['processed']}")
    print(f"Approval rate          : {final['approval_rate']:.1%}")
    print(f"Average risk score     : {final['avg_risk']:.1f}/100")
    print(f"High-risk applications : {final['high_risk']}")
    print(f"Sanctioned exposure    : Rs.{final['exposure']:,}")

    return log_df, final


if __name__ == "__main__":
    run()
```

Element	Explanation
`combine_decision(rule_action, probability)`	The arbitration policy. A HIGH band is an absolute veto — the model cannot overrule it, which is what makes the system defensible to a regulator. A LOW band approves as long as the model is not strongly against it (probability at least 0.40). Everything in between is decided by the model at the 0.50 cut-off.
`class RollingKPI`	Holds running totals instead of recomputing over the whole history. Each `update()` is O(1), so the cost per event stays constant no matter how long the stream runs. This is exactly how streaming aggregations work in Spark Structured Streaming or Flink.
`snapshot()`	Returns the current metrics as a plain dict, cleanly separating *state* from *presentation*. The console printer and the Streamlit dashboard both consume the same snapshot.
`time.sleep(delay)`	The only line that makes this a simulation rather than a batch job. Set `--fast` and it becomes zero; swap the loop for a message consumer and it disappears entirely.
`pd.concat([row, pd.Series(risk)])`	Merges the four risk columns onto the applicant Series so the enriched record carries both the rule output and the raw features into the model call.
Two-layer scoring per record	Rules run first (fast, explainable, vetoing), then the model (slower, statistical). If the model service were ever down, the rule engine alone could keep the bank running — a genuine resilience property, not just a design flourish.
`log.append({...})` then `to_csv`	Builds an audit trail: timestamp, inputs, both scores, the decision and the justification. Financial regulators require this record for every automated credit decision.

10.7 main.py — the orchestrator

Contains no analytics of its own. Its only job is to call the five stages in the right order, print readable banners and write the summary. Keeping the orchestrator thin is what makes each stage independently testable.

SRC/MAIN.PY

```
"""
main.py
-------
Entry point. Runs the five pipeline stages end to end:

    ingest -> risk scoring -> EDA -> model training -> real-time stream

Usage:
    python main.py               # full run, 1 second between stream events
    python main.py --fast        # no delay, useful while debugging
    python main.py --no-charts   # skip matplotlib, console output only
"""

import argparse
from datetime import datetime

import config
import data_loader
import eda
```

```
import model as model_module
import risk_engine
import stream_simulator


def banner(step, title):
    print("\n" + "#" * 68)
    print(f"# STEP {step}: {title}")
    print("#" * 68)


def write_summary(metrics, drivers, stream_kpi, scored):
    """Persist a plain-text report so results survive after the console clears."""
    lines = [
        "BANK LOAN APPROVAL & RISK ANALYTICS - RUN SUMMARY",
        f"Generated : {datetime.now():%Y-%m-%d %H:%M:%S}",
        "",
        "1. DATASET",
        f"   Historical records : {len(scored)}",
        f"   Approval rate      : {(scored[config.TARGET] == 'Y').mean():.1%}",
        f"   Risk band split    : {scored['risk_band'].value_counts().to_dict()}",
        "",
        "2. MODEL",
        f"   Train / test rows  : {metrics['train_rows']} / {metrics['test_rows']}",
        f"   Accuracy           : {metrics['accuracy']}",
        f"   ROC-AUC            : {metrics['roc_auc']}",
        f"   Confusion matrix   : {metrics['confusion_matrix']}",
        "",
        "   Top drivers:",
    ]
    for _, r in drivers.iterrows():
        lines.append(f"     {r['feature']:<40} {r['coefficient']:>8}")

    lines += [
        "",
        "3. REAL-TIME STREAM",
        f"   Applications processed : {stream_kpi['processed']}",
        f"   Approval rate          : {stream_kpi['approval_rate']:.1%}",
        f"   Average risk score     : {stream_kpi['avg_risk']:.1f}/100",
        f"   High-risk applications : {stream_kpi['high_risk']}",
        f"   Sanctioned exposure    : Rs.{stream_kpi['exposure']:,}",
        "",
        "Artefacts written to outputs/: charts (*.png), loan_model.joblib,",
        "realtime_decisions.csv, summary_report.txt",
    ]

    config.SUMMARY_REPORT.write_text("\n".join(lines), encoding="utf-8")
    print(f"\n[main] summary report written to {config.SUMMARY_REPORT.name}")


def main():
    parser = argparse.ArgumentParser(description="Bank Loan Approval & Risk Analytics")
    parser.add_argument("--fast", action="store_true", help="no delay in the stream")
    parser.add_argument("--no-charts", action="store_true", help="skip PNG generation")
    args = parser.parse_args()

    print("\nBANK LOAN APPROVAL & RISK ANALYTICS")
    print(f"Started at {datetime.now():%Y-%m-%d %H:%M:%S}")

    # ---- STEP 1: ingest + feature engineering ------------------------------
    banner(1, "DATA INGESTION & FEATURE ENGINEERING")
    df = data_loader.get_training_data()

    # ---- STEP 2: rule-based risk scoring -----------------------------------
    banner(2, "RULE-BASED RISK SCORING")
    scored = risk_engine.score_dataframe(df)
    print(scored[["application_id", "credit_score", "dti_ratio",
```

```
                "risk_score", "risk_band", "rule_decision",
                config.TARGET]].to_string(index=False))

    # ---- STEP 3: EDA ------------------------------------------------------
    banner(3, "EXPLORATORY DATA ANALYSIS")
    if args.no_charts:
        eda.print_summary(scored)
    else:
        eda.run(scored)

    # ---- STEP 4: model training -------------------------------------------
    banner(4, "MODEL TRAINING & EVALUATION")
    pipe, metrics = model_module.train(scored)
    drivers = model_module.top_drivers(pipe)

    # ---- STEP 5: real-time stream -----------------------------------------
    banner(5, "REAL-TIME APPLICATION STREAM")
    _, stream_kpi = stream_simulator.run(pipe, delay=0 if args.fast else None)

    write_summary(metrics, drivers, stream_kpi, scored)
    print("\nPipeline finished successfully.\n")


if __name__ == "__main__":
    main()
```

Element	Explanation
`argparse`	Adds `--fast` and `--no-charts`. Command-line flags let the same script serve a live demo and a quick debug run without editing code.
`banner(step, title)`	Prints a consistent header before each stage so a viewer following your demo always knows which stage is running.
`write_summary(...)`	Persists the run to `summary_report.txt`. Console output scrolls away; a file can be attached to your submission.
Stage ordering	Risk scoring must precede EDA (the charts plot `risk_band`) and must precede training (`risk_score` is not itself a model input, but the scored frame is what carries every column forward). The stream runs last because it needs the fitted pipeline.

11 How to Run the Project

11.1 Step by step

TERMINAL

```
# 1. unzip the project and open a terminal inside it
cd BankLoanRiskAnalytics

# 2. (recommended) create an isolated environment
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate      # macOS / Linux

# 3. install the five libraries
pip install -r requirements.txt

# 4. run the whole pipeline
cd src
python main.py
```

Command	Effect
`python main.py`	Full run with a 1-second pause between streamed applications. Use this for your demonstration — the pauses make it visibly real-time.
`python main.py --fast`	Same output with no pauses. Finishes in about 3 seconds.
`python main.py --no-charts`	Skips matplotlib entirely. Useful on a machine where the plotting libraries misbehave.

11.2 Running a single module

Every module has a `__main__` block, so you can test any stage in isolation while you work on it:

TERMINAL

```
cd src
python data_loader.py       # ingestion + engineered columns
python risk_engine.py       # rule scores only
python eda.py               # regenerate the four charts
python model.py             # retrain and print coefficients
python stream_simulator.py  # replay the live stream
```

11.3 Optional Streamlit dashboard

TERMINAL

```
pip install streamlit
python src/main.py            # must run once so the model exists
streamlit run app/dashboard.py
```

The dashboard has two tabs: a **live stream** tab that replays the incoming applications with animated KPI tiles, and a **manual scoring** tab where you type in any applicant profile and get an instant decision. It imports the same `src/` modules, so no analytics logic is duplicated.

11.4 Running in Google Colab

COLAB NOTEBOOK CELL

```
from google.colab import files
files.upload()                          # upload the zip
!unzip -q BankLoanRiskAnalytics.zip
%cd BankLoanRiskAnalytics/src
!pip install -q -r ../requirements.txt
!python main.py --fast
```

12 Expected Output

This is the real, unedited console output of `python main.py --fast`. If your run matches this, the project is working correctly. Timestamps and the `Started at` line will naturally differ.

HEADER

```
BANK LOAN APPROVAL & RISK ANALYTICS
Started at 2026-07-28 00:10:57
```

STAGE 1 - INGESTION

```
# STEP 1: DATA INGESTION & FEATURE ENGINEERING

[data_loader] Loaded 20 rows x 15 columns from loan_applications.csv
[data_loader] Validation -> {'rows': 20, 'duplicate_ids': 0, 'missing_values': 0, 'negative_income': 0}
[data_loader] Engineered 5 new features
```

STAGE 2 - RULE SCORING

```
# STEP 2: RULE-BASED RISK SCORING

[risk_engine] Scored 20 applications -> {'LOW': 9, 'HIGH': 8, 'MEDIUM': 3}
application_id  credit_score  dti_ratio  risk_score risk_band rule_decision loan_approved
        LN1001           780      0.330          16       LOW  AUTO-APPROVE             Y
        LN1002           720      0.313          18       LOW  AUTO-APPROVE             Y
        LN1003           610      0.705          88      HIGH   AUTO-REJECT             N
        LN1004           690      0.325          28    MEDIUM MANUAL REVIEW             Y
        LN1005           800      0.361          21       LOW  AUTO-APPROVE             Y
        LN1006           580      0.671         100      HIGH   AUTO-REJECT             N
        LN1007           745      0.344          18       LOW  AUTO-APPROVE             Y
        LN1008           640      0.740          94      HIGH   AUTO-REJECT             N
        LN1009           810      0.347          10       LOW  AUTO-APPROVE             Y
        LN1010           520      1.053         100      HIGH   AUTO-REJECT             N
        LN1011           700      0.507          43    MEDIUM MANUAL REVIEW             Y
        LN1012           665      0.546          76      HIGH   AUTO-REJECT             N
        LN1013           705      0.398          18       LOW  AUTO-APPROVE             Y
        LN1014           760      0.334          10       LOW  AUTO-APPROVE             Y
        LN1015           560      1.056         100      HIGH   AUTO-REJECT             N
        LN1016           730      0.298           8       LOW  AUTO-APPROVE             Y
        LN1017           690      0.424          63      HIGH   AUTO-REJECT             N
        LN1018           600      0.552          83      HIGH   AUTO-REJECT             N
        LN1019           785      0.305          10       LOW  AUTO-APPROVE             Y
        LN1020           655      0.376          33    MEDIUM MANUAL REVIEW             Y
```

STAGE 3 - EXPLORATORY ANALYSIS

```
# STEP 3: EXPLORATORY DATA ANALYSIS


======================================================================
EXPLORATORY DATA ANALYSIS
======================================================================
Records            : 20
Columns            : 24
Approved / Rejected: 12 / 8
Approval rate      : 60.0%

-- Averages by approval status --
               credit_score  annual_income  dti_ratio  loan_to_income  previous_defaults
loan_approved
N                    608.12      742500.00       0.72            2.37              1.25
Y                    740.00      978333.33       0.35            1.23              0.00

-- Approval rate by employment type --
  Salaried        83%
  Self-Employed   29%
  Unemployed      0%
```

```
-- Correlation of numeric drivers with approval --
credit_score         0.798
annual_income        0.239
loan_to_income      -0.764
dti_ratio           -0.782
previous_defaults   -0.913
risk_score          -0.949
[eda] saved 01_approval_distribution.png
[eda] saved 02_credit_score_boxplot.png
[eda] saved 03_income_vs_loan.png
[eda] saved 04_risk_bands.png
```

STAGE 4 - MODEL TRAINING

```
# STEP 4: MODEL TRAINING & EVALUATION


======================================================================
MODEL TRAINING
======================================================================
Train rows : 14   Test rows : 6
Accuracy   : 0.833
ROC-AUC    : 1.0
Confusion matrix [[TN FP] [FN TP]]:
[[1 1]
 [0 4]]

Classification report:
              precision    recall  f1-score   support

    Rejected       1.00      0.50      0.67         2
    Approved       0.80      1.00      0.89         4

    accuracy                           0.83         6
   macro avg       0.90      0.75      0.78         6
weighted avg       0.87      0.83      0.81         6

[model] pipeline saved to loan_model.joblib

Top model drivers (positive = pushes towards approval):
               feature  coefficient
     num__credit_score        0.678
num__previous_defaults       -0.665
   num__loan_to_income       -0.532
        num__dti_ratio       -0.512
   num__monthly_income        0.382
    num__annual_income        0.382
     num__existing_emi       -0.290
      cat__gender_Male        0.223
```

STAGE 5 - REAL-TIME STREAM

```
# STEP 5: REAL-TIME APPLICATION STREAM

[data_loader] Loaded 8 rows x 14 columns from incoming_stream.csv
[data_loader] Validation -> {'rows': 8, 'duplicate_ids': 0, 'missing_values': 0, 'negative_income': 0}
[data_loader] Engineered 5 new features

======================================================================
REAL-TIME DECISION STREAM
======================================================================

[00:10:58] LN2001 | Harish Bhat
    income Rs.1,150,000/yr | loan Rs.1,500,000 | CIBIL 770 | DTI 28%
    risk 0/100 (LOW) | model P(approve)=0.97
    DECISION: APPROVED  <- low risk + model agrees
    reasons : no risk flags
    LIVE KPI: processed=1 approval=100% avg-risk=0.0 high-risk=0 exposure=Rs.1,500,000

[00:10:58] LN2002 | Kavya Raman
    income Rs.390,000/yr | loan Rs.800,000 | CIBIL 610 | DTI 65%
```

```
    risk 83/100 (HIGH) | model P(approve)=0.12
    DECISION: REJECTED  <- rule veto: high risk band
    reasons : credit score 610 (+30); DTI 65% (+35); loan is 2.1x income (+6); 1 previous default(s) (+12)
    LIVE KPI: processed=2 approval=50% avg-risk=41.5 high-risk=1 exposure=Rs.1,500,000

[00:10:58] LN2003 | Imran Sheikh
    income Rs.1,750,000/yr | loan Rs.2,600,000 | CIBIL 735 | DTI 31%
    risk 23/100 (LOW) | model P(approve)=0.94
    DECISION: APPROVED  <- low risk + model agrees
    reasons : credit score 735 (+8); DTI 31% (+10); self-employed income (+5)
    LIVE KPI: processed=3 approval=67% avg-risk=35.3 high-risk=1 exposure=Rs.4,100,000

[00:10:58] LN2004 | Divya Kulkarni
    income Rs.700,000/yr | loan Rs.1,400,000 | CIBIL 655 | DTI 58%
    risk 71/100 (HIGH) | model P(approve)=0.18

    ... applications LN2004 to LN2007 follow the same format ...

    risk 88/100 (HIGH) | model P(approve)=0.07
    DECISION: REJECTED  <- rule veto: high risk band
    reasons : credit score 625 (+30); DTI 64% (+35); loan is 2.4x income (+6); 1 previous default(s) (+12); self-
employed income (+5)
    LIVE KPI: processed=7 approval=43% avg-risk=52.1 high-risk=4 exposure=Rs.5,000,000

[00:10:58] LN2008 | Swathi Naidu
    income Rs.560,000/yr | loan Rs.500,000 | CIBIL 715 | DTI 32%
    risk 18/100 (LOW) | model P(approve)=0.89
    DECISION: APPROVED  <- low risk + model agrees
    reasons : credit score 715 (+8); DTI 32% (+10)
    LIVE KPI: processed=8 approval=50% avg-risk=47.9 high-risk=4 exposure=Rs.5,500,000

[stream] decision log written to realtime_decisions.csv

--- SESSION SUMMARY ---
Applications processed : 8
Approval rate          : 50.0%
Average risk score     : 47.9/100
High-risk applications : 4
Sanctioned exposure    : Rs.5,500,000

[main] summary report written to summary_report.txt

Pipeline finished successfully.
```

12.1 Files produced

File in outputs/	Size	Contents
01_approval_distribution.png	~18 KB	Approved vs rejected bar chart
02_credit_score_boxplot.png	~20 KB	Credit score distribution per outcome
03_income_vs_loan.png	~46 KB	Income vs loan size bubble scatter
04_risk_bands.png	~19 KB	Count of LOW / MEDIUM / HIGH applications
loan_model.joblib	~5 KB	The fitted scikit-learn pipeline
realtime_decisions.csv	~1.4 KB	One audit row per streamed application
summary_report.txt	~1.1 KB	Plain-text KPIs for the whole run

13 Results and Interpretation

13.1 What the charts show

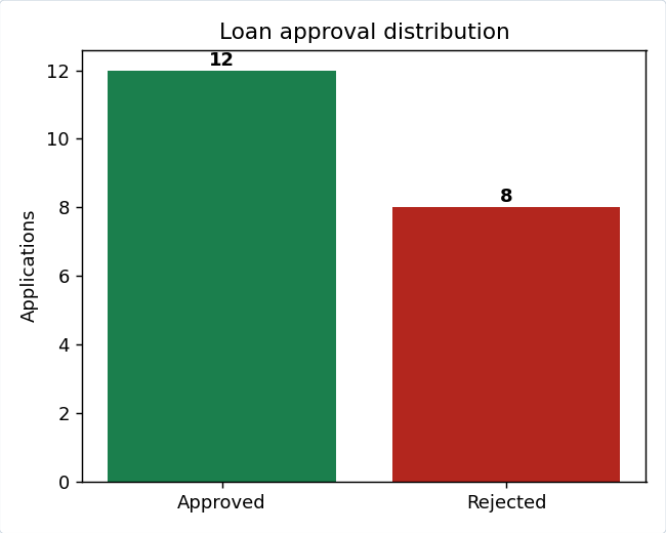


Fig 1 — Approval distribution

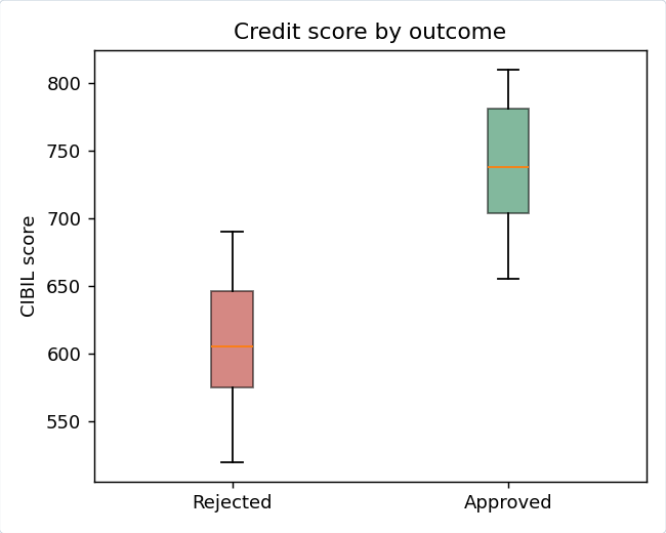


Fig 2 — Credit score by outcome

Figure 1. Twelve approvals against eight rejections, a 60% approval rate. Mild imbalance — enough to justify `class_weight="balanced"`, not enough to need resampling.

Figure 2. The two boxes barely overlap. Approved applicants cluster around 700–810; rejected ones around 520–690. Credit score alone would classify most of this dataset correctly, which matches its +0.798 correlation with approval.

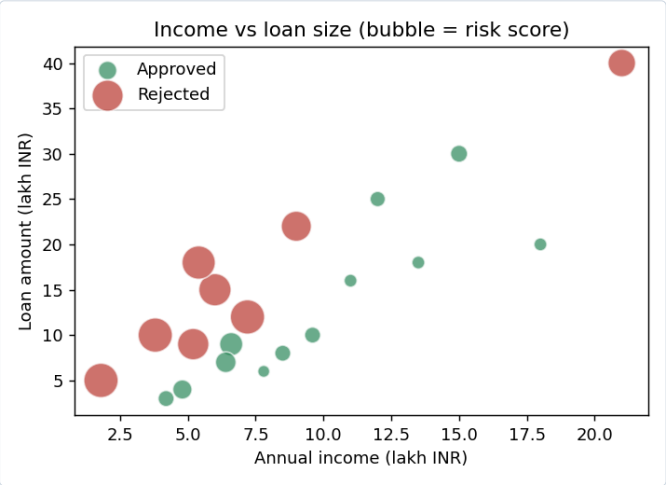


Fig 3 — Income vs loan size

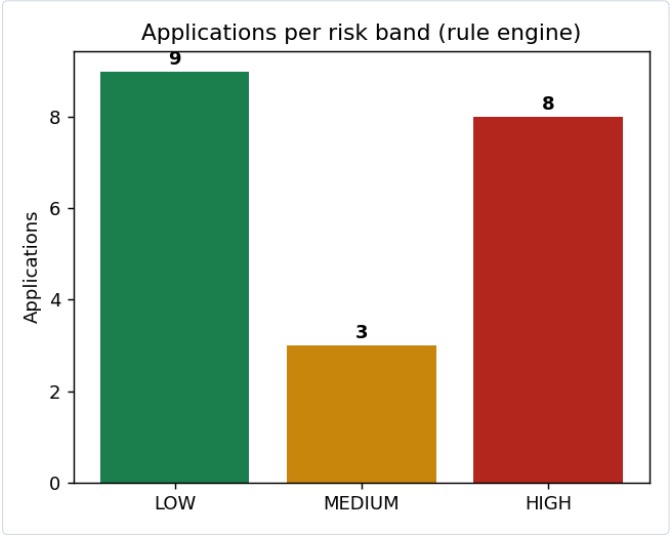


Fig 4 — Risk band split

Figure 3. Bubble size is the risk score. Rejected applications sit above the diagonal — they ask for a lot relative to what they earn. Notice LN1017: the highest income in the dataset, still rejected, because a Rs.40 lakh request against Rs.30,000 of existing EMIs and one prior default is not affordable. **Income alone never decides a loan.**

Figure 4. Nine LOW, three MEDIUM, eight HIGH. Only the three MEDIUM files would reach a human officer — an 85% straight-through processing rate, which is the operational payoff of the whole system.

13.2 Model performance

Metric	Value	How to read it
Training rows	14	70% of 20, stratified
Test rows	6	30% held out, never seen during fitting
Accuracy	0.833	5 of 6 test applications classified correctly
ROC-AUC	1.000	Perfect *ranking*: every approved applicant received a higher probability than every rejected one. The one error came from where the 0.50 cut-off sits, not from the ordering.
Confusion matrix	[[1, 1], [0, 4]]	1 true negative, 1 false positive, 0 false negatives, 4 true positives.
Precision (Rejected)	1.00	Everything the model called a rejection really was one.
Recall (Approved)	1.00	No creditworthy applicant was turned away.

Do not celebrate an AUC of 1.0

With six test rows, a single reshuffle could move accuracy by 17 percentage points. These numbers demonstrate that the *pipeline is correct*; they say almost nothing about how the model would perform on real applications. Honest reporting of this limitation is worth more marks than a big number. Section 17 shows how to get a defensible estimate with cross-validation.

13.3 What drives the decision

Feature	Coefficient	Direction and meaning
`credit_score`	+0.678	Strongest push towards approval, as expected.
`previous_defaults`	−0.665	Almost equally strong push towards rejection. Past behaviour predicts future behaviour.
`loan_to_income`	−0.532	Asking for too much relative to earnings hurts.
`dti_ratio`	−0.512	An engineered feature ranking fourth — proof that feature engineering, not the algorithm, created most of the value here.
`annual_income`	+0.382	Helps, but is beaten by all three ratio features.
`existing_emi`	−0.290	Existing commitments reduce headroom.
`gender_Male`	+0.223	An artefact of 20 rows, not a real effect. Flagged in 13.4.

13.4 A fairness observation you should mention in your viva

`gender_Male` appears among the top drivers with a positive coefficient. In a dataset of twenty rows this is pure noise — but in a real lending model it would be **illegal in most jurisdictions**. Protected attributes such as gender, marital status, religion and caste must not influence a credit decision, directly or through proxies.

The correct fix is one line: remove `"gender"` and `"marital_status"` from `config.CATEGORICAL` and rerun. Do it, compare the accuracy, and report the result — noticing this unprompted is exactly the kind of judgement that separates a good project from a routine one.

13.5 Real-time session results

Eight applications were streamed. Four were approved and four rejected, committing Rs.55,00,000 of exposure at an average risk score of 47.9/100.

ID	Applicant	CIBIL	DTI	Risk	Band	P(approve)	Decision
LN2001	Harish Bhat	770	28%	0	LOW	0.968	APPROVED
LN2002	Kavya Raman	610	65%	83	HIGH	0.117	REJECTED
LN2003	Imran Sheikh	735	31%	23	LOW	0.938	APPROVED
LN2004	Divya Kulkarni	655	58%	71	HIGH	0.175	REJECTED
LN2005	Naveen Prasad	540	133%	100	HIGH	0.0	REJECTED
LN2006	Ayesha Fernandes	795	27%	0	LOW	0.962	APPROVED
LN2007	Rohit Chauhan	625	64%	88	HIGH	0.071	REJECTED
LN2008	Swathi Naidu	715	32%	18	LOW	0.895	APPROVED

Every rejection was a rule-engine veto, and in each case the model independently agreed (probabilities of 0.12, 0.18, 0.00 and 0.07). The two layers converging on the same answer is the ideal outcome: the model is learning the same economics the rules encode, and the rules are giving the customer a sentence the model could not.

14 Testing Checklist

Work down this list before you submit or demonstrate. Each row is something a reviewer might actually try.

#	Test	Expected result
T1	`python data_loader.py`	Prints 20 rows loaded, a validation dict with all zeros, and 5 engineered features
T2	Delete `data/loan_applications.csv` and run	`FileNotFoundError` naming the exact missing path, not a stack trace deep inside pandas
T3	Blank out a `credit_score` cell in the CSV and run	`missing_values: 1` in the report, then the median filled in and the run continues
T4	Duplicate a whole row in the CSV and run	`duplicate_ids: 1` and only 20 rows survive
T5	Hand-check LN1001's EMI with a calculator	About Rs.24,959, matching `estimated_emi`
T6	Verify LN2005's risk score by adding the penalties in Section 9.6	129 raw, capped to 100, band HIGH
T7	`python risk_engine.py`	Band counts of LOW 9, MEDIUM 3, HIGH 8
T8	Compare `rule_decision` against `loan_approved` for all 20 rows	Every AUTO-REJECT corresponds to an actual N — the rulebook is consistent with history
T9	`python eda.py`	Four PNG files appear in outputs/
T10	`python model.py` twice	Identical accuracy both times, proving `random_state` works
T11	Delete `outputs/loan_model.joblib` then run `python stream_simulator.py`	A clear `FileNotFoundError` telling you to run main.py first
T12	Set `APPROVAL_THRESHOLD = 0.80` in config and rerun	Fewer approvals; the arbitration logic responds to configuration alone
T13	Add a new row to `incoming_stream.csv`	It is scored automatically, with no code change
T14	Change `loan_purpose` to an unseen value such as `Wedding`	No crash — `handle_unknown="ignore"` handles it
T15	Open `outputs/realtime_decisions.csv`	8 rows, each with a decision and a non-empty justification

15 Common Errors and Fixes

Error message	Cause	Fix
`ModuleNotFoundError: No module named 'config'`	You ran the script from the project root instead of from inside `src/`	`cd src` first, then `python main.py`
`ModuleNotFoundError: No module named 'sklearn'`	Dependencies not installed, or installed into a different interpreter	`pip install -r requirements.txt`; on some systems use `python -m pip` to be sure it is the same Python
`FileNotFoundError: ...loan_applications.csv`	The data folder was moved or renamed	Keep the folder structure from the zip intact
`FileNotFoundError: Model not trained yet`	You ran `stream_simulator.py` before ever training	Run `python main.py` once so the joblib file is created
`ValueError: The least populated class in y has only 1 member`	You edited the CSV and left fewer than two rows of one class	Keep at least two Y and two N rows so stratification can work
`UserWarning: FixedFormatter should only be used...`	A matplotlib version difference in axis labelling	Harmless; it does not affect the charts
Charts are blank or the script hangs	An interactive matplotlib backend is being used	`matplotlib.use("Agg")` is already set in `eda.py` — make sure it stays above the pyplot import
`TypeError: boxplot() got an unexpected keyword argument 'tick_labels'`	matplotlib older than 3.9	Either upgrade matplotlib or rename the argument to `labels=`
Rupee or unicode symbols show as boxes in the console	Windows console code page	The code deliberately uses `Rs.` instead of the rupee glyph; keep it that way
Streamlit page loads but is empty	The model file does not exist yet	Run `python src/main.py` before launching the dashboard

16 Viva / Interview Questions with Answers

These are the questions most likely to be asked about this project. Read the answers, then close the handbook and try to say them in your own words.

Q1. Why did you build both a rule engine and a machine learning model?

They fail in opposite ways. Rules are transparent and work from day one but cannot capture interactions between variables. A model captures those interactions but cannot explain itself to a customer or a regulator. Running both, and letting the rules veto the extremes, gives explainable safety on the boundaries and statistical accuracy in the grey zone.

Q2. What exactly is real-time about this project?

The processing model. Records are handled one at a time as they arrive, each one produces a decision immediately, and the aggregate KPIs update after every event rather than at the end. The CSV plus timer stands in for a message queue, but the event loop, the per-record scoring path and the O(1) rolling aggregation are the same design a production system uses.

Q3. Why Logistic Regression rather than Random Forest or XGBoost?

Three reasons. It is stable on 20 rows where a tree ensemble would overfit badly. It outputs a calibrated probability the business can threshold. And its coefficients are directly interpretable, which credit regulation in most countries effectively requires.

Q4. What is debt-to-income and why does it dominate the scorecard?

DTI is total monthly EMI divided by monthly income. It measures what actually causes default: not being poor, but having no cash left after servicing debt. In this dataset approved applicants average 0.35 and rejected ones 0.72.

Q5. Why use a Pipeline instead of transforming the data first?

To eliminate training/serving skew. The Pipeline stores the scaler's means and the encoder's categories learned during fitting and replays exactly those at prediction time. Transforming manually, then saving only the estimator, is the single most common way deployed models silently break.

Q6. Why stratify the train/test split?

To preserve the 60/40 class ratio in both halves. With 20 rows, a random split could easily put all rejections into training, leaving a test set that cannot measure anything.

Q7. Your ROC-AUC is 1.0. Is the model perfect?

No. AUC measures ranking, and on six test rows every approved applicant happened to outrank every rejected one. The accuracy was still 0.833 because of where the 0.50 cut-off falls. With this sample size the correct statement is that the pipeline is validated, not that the model is accurate. Repeated stratified cross-validation would be the honest next step.

Q8. Why is median used for numeric imputation but mode for categorical?

The median is unaffected by extreme values, so one very high income cannot distort the fill value the way a mean would. Categories have no ordering, so the only sensible summary is the most frequent value.

Q9. What does class_weight="balanced" actually do?

It scales each class's contribution to the loss function inversely to its frequency, so the 8 rejections carry as much total weight as the 12 approvals. Without it the model is nudged towards always predicting the majority class.

Q10. How would you deploy this?

Wrap predict_one in a FastAPI POST endpoint, containerise it with Docker, replace the CSV reader with a Kafka consumer, write decisions to Postgres instead of CSV, push KPIs to Prometheus and visualise in Grafana. The src/ modules themselves would need almost no change — which is the point of the layered design.

Q11. How would you detect that the model has gone stale?

Monitor drift. Track the distribution of incoming features against the training distribution using population stability index, watch the approval rate for sudden shifts, and once repayment outcomes arrive, recompute AUC on recent cohorts. Retrain on a schedule and whenever drift crosses a threshold.

Q12. What are the ethical risks in this system?

Two main ones. First, protected attributes: gender appears as a driver here purely because of the small sample, but using it in production would be discriminatory and in most places unlawful; the same applies to proxies such as postcode. Second, feedback loops: the model is trained on past approvals, so any historical bias in who was approved gets learned and amplified. Both are why the explainable rule layer and human review of the MEDIUM band matter.

Q13. Why is the risk score capped at 100?

To keep the scale interpretable and the charts bounded. LN2005 accumulates 129 raw penalty points; without the cap the 0–100 axis and the band logic would both become meaningless.

Q14. What does the confusion matrix [[1, 1], [0, 4]] tell you?

Reading [[TN, FP], [FN, TP]]: one rejection correctly identified, one rejection wrongly approved (a false positive, the costly error for a bank), no creditworthy applicant wrongly refused, and four correct approvals. For a lender, false positives cost money directly, so you would raise the approval threshold above 0.50 to trade recall for precision.

Q15. If you had 100,000 rows instead of 20, what would change?

Add repeated stratified k-fold cross-validation, compare Logistic Regression against gradient boosting, tune hyper-parameters with a randomised search, calibrate probabilities, add SHAP values for per-record explanations, and split the data by time so the test set is genuinely in the future relative to training.

17 Extension Ideas

Ordered from a single evening's work to a full second project. Pick two or three and you have a genuinely distinctive submission.

Level	Extension	What it teaches
Easy	Remove `gender` and `marital_status` from the feature list and compare accuracy	Fairness-aware modelling and ablation testing
Easy	Add 30 more rows to the training CSV and rerun	How sample size changes both the metrics and their stability
Easy	Add a fifth chart: risk score plotted against approval probability	Rule/model agreement analysis
Easy	Print a per-record processing latency in milliseconds	Throughput measurement in streaming systems
Medium	Replace the single split with `RepeatedStratifiedKFold` and report a mean accuracy with a standard deviation	Honest evaluation on small datasets
Medium	Add a `DecisionTreeClassifier` and a `RandomForestClassifier` and compare all three	Model selection and the bias/variance trade-off
Medium	Log every decision into SQLite instead of CSV, with a queryable schema	Persistence and basic data engineering
Medium	Expose `predict_one` through a FastAPI `/score` endpoint and call it with curl	Turning a model into a service
Medium	Add loan pricing: derive an interest rate from the risk score instead of a fixed 10.5%	Risk-based pricing, how lenders actually make money
Hard	Implement drift detection: compare the streamed feature distribution against the training distribution using PSI and raise an alert	Production ML monitoring
Hard	Add SHAP explanations so every model prediction gets its own feature attribution	Explainable AI beyond linear coefficients
Hard	Replace the CSV reader with a real Kafka consumer running in Docker	Genuine stream processing infrastructure
Hard	Build a champion/challenger framework that runs two models side by side and routes a percentage of traffic to each	Safe model deployment and A/B testing

18 Evaluation Rubric

A suggested 100-mark scheme. Useful for instructors, and useful for students as a self-check before submitting.

Criterion	Marks	What full marks looks like
Setup and execution	10	The project runs end to end from a clean environment using only the instructions in the README
Data handling	10	Validation, imputation and duplicate handling all demonstrated; the student can explain why median and mode were chosen
Feature engineering	15	All five derived features explained, and the EMI or DTI calculation reproduced by hand for one record
Rule engine	15	The rulebook is explained, one score is derived manually, and the student can justify each threshold
Exploratory analysis	10	All four charts interpreted, with correlations linked back to domain reasoning
Modelling	15	Pipeline, scaling, encoding, stratification and metrics explained; coefficients interpreted correctly
Real-time implementation	15	The event loop, the arbitration policy and the rolling KPIs are explained, and the demo visibly processes records one at a time
Critical thinking	10	Limitations of a 20-row dataset acknowledged, the gender coefficient flagged as a fairness issue, and at least one extension attempted

Where marks are most often lost

Not in the code — the code is given to you. Marks are lost when a student cannot explain *why* a design choice was made: why a Pipeline instead of manual transforms, why stratify, why the rule engine can veto the model. Section 16 covers every one of those questions.

19 Glossary

Term	Meaning
Credit score / CIBIL	A bureau-issued number from 300 to 900 summarising an individual's repayment history. Above 750 is considered excellent.
EMI	Equated Monthly Instalment — the fixed monthly payment that repays principal and interest over the loan term.
DTI	Debt-to-Income ratio: total monthly EMI divided by monthly income. The core affordability measure.
LTI	Loan-to-Income: loan amount divided by annual income, expressed as a multiple.
Exposure	The total amount of money a lender has committed and could lose.
Default	Failure to repay according to the agreed schedule.
Straight-through processing	An application decided entirely by software, with no human intervention.
Feature engineering	Creating new input variables from raw data in order to expose the underlying signal to a model.
Pipeline (scikit-learn)	An object chaining preprocessing steps and an estimator so that all of them are fitted and applied together.
One-hot encoding	Converting a categorical column into a set of binary columns, one per category.
Standardisation	Rescaling a numeric column to mean 0 and standard deviation 1 so that variables on different scales become comparable.
Stratified split	A train/test split that preserves the class proportions of the original dataset.
Confusion matrix	A 2×2 table of true negatives, false positives, false negatives and true positives.
ROC-AUC	Area under the receiver operating characteristic curve; the probability that a randomly chosen positive is ranked above a randomly chosen negative.
Precision / Recall	Precision is how many predicted positives were correct; recall is how many actual positives were found.
Class imbalance	A dataset where one outcome is much more frequent than the other.
Training/serving skew	A mismatch between how data is prepared during training and during prediction; a leading cause of silent model failure.
Model drift	Degradation of model performance over time as the real-world data distribution moves away from the training distribution.
Rolling aggregate	A statistic updated incrementally as each new event arrives, rather than recomputed from scratch.
Audit trail	A permanent, timestamped record of every automated decision and the reasoning behind it, required by financial regulators.
Champion / challenger	An architecture in which an established decision system and a new one run in parallel so the new one can be evaluated safely.

End of handbook — Bank Loan Approval & Risk Analytics.
Everything described here is included in the accompanying zip file. Run it, break it, extend it.